

1. INTRODUCTION

The Student Record Management System is a menu-driven Python application developed to manage student information efficiently.

The system allows users to add, view, search, update, and delete student records.

It demonstrates key programming concepts such as file handling, exception handling, input validation and logging.

The system stores basic student information in a CSV file (students.csv).

And additional details such as address and contact in a JSON file (students.json).

2. SYSTEM DESIGN

The system is designed using a modular approach where each functionality is implemented in a separate function.

The main menu controls user interaction and repeatedly prompts the user until they choose to exit.

Key Components include:

--CSV File (students.csv)

Stores basic student details:

Registration Number

Name

Program

--JSON File (students.json)

Stores additional student details:

Address

Contact

Program

--Log File (student_system.log)

Records all system activities and errors for tracking and debugging.

3. PROGRAM FEATURES:

--Add Student:

Allows the user to enter student details. The data is saved in both CSV and JSON files.

Input validation ensures that required fields are not empty and contact numbers are valid.

--View students:

Reads all records from the CSV file and displays them in a user-friendly format.

--Search students:

Searches for a student using the registration number.

If the student is not found, a custom exception (StudentNotFoundError) is raised.

--Update students:

Updates an existing student's name, program, address and contact information.

The changes are reflected in both CSV and JSON files.

--Delete students:

Removes a student record from both CSV and JSON files based on the registration number.

4.EXCEPTION HANDLING STRATEGY:

The system uses robust error handling techniques:

--try block: Wraps code that may cause errors.

--except block: Handles exceptions such as file errors and missing students.

--finally block: Executes cleanup messages after each operation.

Custom Exception:

A custom exception called StudentNotFoundError is used to handle cases where a student does not exist in the system.

5.LOGGING SYSTEM:

The logging module is used to track system activities and errors. All major actions such as adding, updating,

deleting and viewing students are logged into student_system.log.

Log format:

%(asctime)s - %(levelname)s - %(message)s. This helps in debugging and maintaining system records.

6.TESTING AND RESULTS:

The system was tested using multiple student records.

****Test Cases:**

- Adding students → Successfully stored in CSV and JSON
- Viewing students → Displays all records correctly
- Searching students → Returns correct student or error message
- Updating students → Successfully modifies records
- Deleting students → Removes records from system
- Invalid input → Proper error messages displayed

*****Sample Output:**

- Student records are displayed in a readable format.
- Invalid menu choices are handled correctly.
- System exits safely when option 6 is selected.

7.CHALLENGES:

Managing two file formats (CSV and JSON) simultaneously.

Ensuring data consistency between files.

Handling missing student records.

8.CONCLUSION:

The Student Record Management System successfully demonstrates the use of Python file handling, exception handling, logging and modular programming.

It provides a simple but effective way to manage student records and can be extended into a more advanced database-driven system in the future.